

Introduction to R

Institute of Finance and Statistics

Contents

Preface	5
1 The first steps in R	7
1.1 RStudio for the first time	7
1.2 Simple calculations	8
1.3 Assignments and matrix algebra	9
1.4 Operators	13
1.5 Further data objects	15
1.6 R help	15
1.7 R packages	16
1.8 Exercises I	16
2 Create and load data sets	19
2.1 Simulated data	19
2.2 Load datasets	20
2.3 Save datasets	21
2.4 Exercises II	21
3 Simple programming	23
3.1 The for-loop	23
3.2 The if-condition	24
3.3 The while-loop	25
3.4 Time saving and loops	25
3.5 Exercises III	26
4 Functions	29
4.1 Function definition	29
4.2 Recursion	30
4.3 Exercises IV	31
5 Plotting in R	33
5.1 The plot-function	33
5.2 Histograms and kernel densities	39
5.3 Exercises V	44

6	Monte Carlo simulations	45
6.1	Some important concepts	45
6.2	Simple simulation example	46
6.3	Consistency	47
6.4	Asymptotic normality	50
6.5	Testing	52
6.6	Exercises VI	54
7	Regression analysis	57
7.1	Basic linear model	57
7.2	Empirical example	57
7.3	Exercises VII	61
8	Some notes on useful packages	63
8.1	The tidyverse family	63
8.2	Tidymodels	63
8.3	lmtest	63
8.4	MASS	64
8.5	AER - “Applied Econometrics with R”	64
8.6	knitr	64
8.7	Shiny	64
8.8	Faster Code	64

Preface

The purpose of this tutorial is to introduce the software package R that is used during the lectures, in problem sets, and in the tutorials. The pdf version of these lecture notes is on the class website. The html version is better formatted and can be found [here](#).

Some additional references and tutorials you might find helpful are:

- The online textbook Introduction to Econometrics with R
- Reference card for R commands
- Matlab/R reference
- The official introduction to R
- www.r-project.org
- An R-package for learning R: www.swirl.com
- A book project which covers also advanced issues such as “writing performant code” and “package development”: adv-r.had.co.nz
- Another excellent book: R for Data Science
- An introduction to statistical learning with R

Here are also links to more advanced tutorials:

- Introduction to data science
- Creating dynamic graphics
- ChatGPT

Why do we use R?

- R is free of charge and can be downloaded from <https://cran.rstudio.com/>
- The IDE RStudio for R is also free of charge and can be downloaded from <https://rstudio.com/products/rstudio/download/#download>

- R is equipped with one of the most flexible and powerful graphic routines available anywhere. For instance, check out one of the following repositories:
 - Clean Graphs
 - Publication ready plots
- Today, R is the de-facto standard for statistical science.

Chapter 1

The first steps in R

1.1 RStudio for the first time

I highly recommend to install RStudio. For installation just follow the instructions on

<https://rstudio.com/products/rstudio/download/#download>

When you open RStudio, it is typically split in 4 panels as you can see in the figure below:

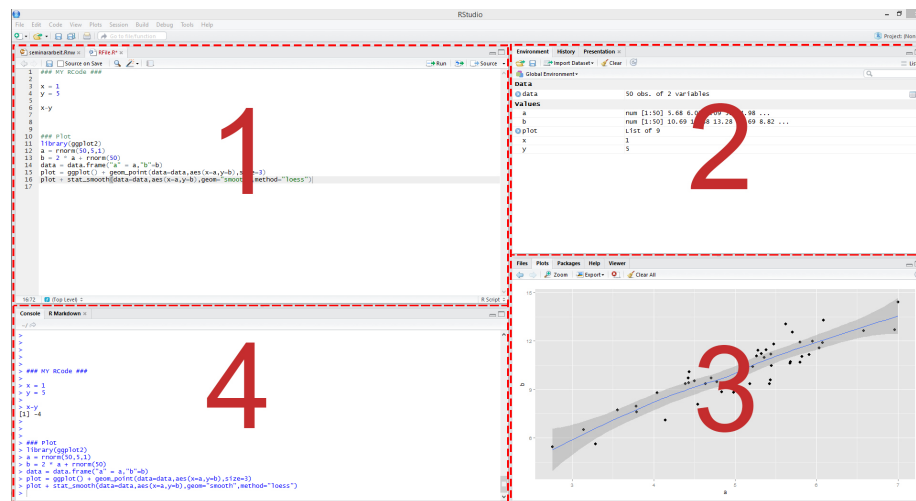


Figure 1: Illustration of RStudio

These panels show:

1. Your script file, which is an ordinary text file with suffix “R”. For instance, yourFavoriteFileName.R. This file contains your code.

2. All objects we have defined in the environment.
3. Help files and plots.
4. The console, which allows you to type commands and display results.

Some other terms that will be useful:

- Working directory: The file-directory you are working in. Useful commands: with `getwd()` you get the location of your current working directory and `setwd()` allows you to set a new location for it. This is very useful when you would like to load or save several files. You can just set your working directory globally and all files can be loaded or will be saved into this directory.
- Workspace: This is a hidden file (stored in the working directory), where all objects you use (e.g., data, matrices, vectors, variables, functions, etc.) are stored. Useful commands: `ls()` shows all elements in our current workspace and `rm(list=ls())` deletes all elements in our current workspace. It is also possible to remove only some objects with `rm(object1, object2)`.

1.2 Simple calculations

We start with a very basic calculation. Let's type `1+1` into the console window and press enter. We get

```
1+1 # first calculation
```

```
## [1] 2
```

i.e. the result of the calculation is returned.

Note that everything that is written after the `#`-sign is ignored by R, which is very useful to comment your code. The second window above, starting with `##`, shows the output.

Let's consider a second calculation

```
1+2+3+ # second calculation
```

```
## Error: <text>:2:0: unexpected end of input
## 1: 1+2+3+ # second calculation
##      ^
```

This calculation ended with an error. The reason is that the command can only be processed if entered correctly.

```
1+2+3+4 # second calculation
```

```
## [1] 10
```

A similar thing happens when parentheses are not closed.

```
2*(2+3
```

```
## Error: <text>:2:0: unexpected end of input
## 1:  2*(2+3
##      ^
```

It will be helpful to use a script file, such as `yourFavoritFileName.R`, to store your R commands. Otherwise, you would have to type your code again when an error occurred. You can send single lines or marked regions of your R-code to the console by pressing the keys **STRG+ENTER**.

1.3 Assignments and matrix algebra

The **assignment** operator will be your most often used tool. Here is an example where a **scalar** variable is created:

```
x <- 9
x
```

```
## [1] 9
```

```
x+1
```

```
## [1] 10
```

Note: The `<-` assignment operator is an unusual syntax that you do not find in other programming languages. In many cases you can alternatively use the `=` operator. Typical R style uses the `<-` assignment operator due to some more in depth-differences and compatibility with older version of R. More on R style recommendations can be found in the tidyverse style guide. In simple settings, alternatives to `<-` are

```
x = 9
x
```

```
## [1] 9
```

or even

```
4 -> y # possible but unusal
y
```

```
## [1] 4
```

We consider now a vector, an object you will use frequently:

```
z <- c(1,3,5,6)
z
```

```
## [1] 1 3 5 6
```

As discussed, `ls()` states the content of the workspace, whereas `rm(list = ls())` deletes the workspace.

```
ls()

## [1] "all_eps"          "b0"                "b1"
## [8] "e"                "E"                "eps"
## [15] "lm_result"        "lm_result_rating" "lm_summary"
## [22] "my_dens"          "my_function"      "my_hc_vcov"
## [29] "myFirst.Dataframe" "myFirst.List"     "n"
## [36] "results"          "results2"         "results3"
## [43] "theta"            "theta_hat"        "time"
## [50] "v"                "visualise.data"   "x"

rm(list=ls())
ls()
```

```
## character(0)
```

In the next step we will consider some vector multiplication. There are three different ways to multiply a vector, namely element by element, using the inner product, or using the outer product. Element by element gives you a vector of the same dimension.

```
z <- c(1,3,5,6)
z*z                #multiplication element by element
```

```
## [1] 1 9 25 36
```

The function `t()` gives the transpose of a vector (or matrix). It is therefore tempting to calculate the inner product as

```
t(z)*z            #multiplication inner product
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    9   25   36
```

but this does not give the desired answer. Instead, you have to use

```
z <- c(1,3,5,6)
t(z)%*%z          #multiplication inner product
```

```
##      [,1]
## [1,]    71
```

Note that R stores zTz as a matrix.

```
class(t(z)%*%z)
```

```
## [1] "matrix" "array"
```

```
class(z)
```

```
## [1] "numeric"
```

Finally, we have the outer product that gives us a matrix

```
z <- c(1,3,5,6)
z%*%t(z)                #multiplication outer product
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    3    5    6
## [2,]    3    9   15   18
## [3,]    5   15   25   30
## [4,]    6   18   30   36
```

In general, be very careful with `%*%` versus `*`. They are often both feasible, but yield different results!

You can also multiply a vector with a scalar:

```
z*4
```

```
## [1]  4 12 20 24
```

A strength of R is that operations can be conducted for each element of the vector in one statement

```
z^2
```

```
## [1]  1  9 25 36
```

```
log(z)
```

```
## [1] 0.000000 1.098612 1.609438 1.791759
```

```
(z-mean(z))/sd(z)
```

```
## [1] -1.2402159 -0.3382407  0.5637345  1.0147221
```

So far, we have seen the function `c()` that can be used to combine objects. All function calls use the same general notation: a function name is always followed by round parentheses. Sometimes, the parentheses include arguments, such as

```
z <- seq(from = 1, to = 5, by = 1)
z
```

```
## [1] 1 2 3 4 5
```

With square brackets you can access particular elements, such as

```
z[1:3]                #refer to particular elements
```

```
## [1] 1 2 3
```

We will next define a matrix and perform matrix-vector multiplication.

```
M <- matrix(data=1:9, nrow=3, ncol=3)    # define matrix M
M
```

```
##      [,1] [,2] [,3]
```

```
## [1,] 1 4 7
## [2,] 2 5 8
## [3,] 3 6 9
```

```
dim(M) # dimension of M
```

```
## [1] 3 3
```

```
dim(z) # dimension of z - does not work because z is a vector
```

```
## NULL
```

```
length(z) # number of elements of z
```

```
## [1] 5
```

Element-by-element multiplication for M is given by

```
M * M
```

```
##      [,1] [,2] [,3]
## [1,] 1 16 49
## [2,] 4 25 64
## [3,] 9 36 81
```

Standard matrix multiplication is again done using `%%` such as

```
M %% M
```

```
##      [,1] [,2] [,3]
## [1,] 30 66 102
## [2,] 36 81 126
## [3,] 42 96 150
```

```
t(M) %% M
```

```
##      [,1] [,2] [,3]
## [1,] 14 32 50
## [2,] 32 77 122
## [3,] 50 122 194
```

If matrices and vectors have suitable dimensions, you can multiply them as follows:

```
M %% z[1:3]
```

```
##      [,1]
## [1,] 30
## [2,] 36
## [3,] 42
```

Similar to before, with square brackets you can excess particular elements, such as

```
M[,3]           # state the third column of M

## [1] 7 8 9
M[3,3]         # state element in the third column and third row of M

## [1] 9
```

Here are some more examples:

```
M[,1]           # first column of M
M[3,3]         # element in the third column and third row of M
z[c(1,4)]      # first and fourth element of z
```

Logical statements are also very useful. For example, if you want to know which elements in the first column of matrix M are strictly greater than 2:

```
# See which elements:
M[,1]>2

## [1] FALSE FALSE TRUE
# and extract the elements
M[M[,1]>2,1]
```

```
## [1] 3
```

Note: Logical operations return so-called *boolean* objects, i.e., either a **TRUE** or a **FALSE**. For instance, if we ask R whether `1>2` we get the answer **FALSE**.

1.4 Operators

We have already seen basic arithmetic operations such as

```
+, -, *, /
```

Decimals are declared with a “.”, not a comma

```
1.8 not 1,8
```

Exponents are declared with ^, i.e. 2^3 is written as

```
2^3
```

For mathematical calculations, only parentheses () can be used.

Frequently used mathematical functions are

```
sqrt()         # square root
exp()          # exponential function
log()          # nat. Logarithm
log(...,10)    # Logarithm with Base 10
abs()          # absolute value
```

```

round(...,x)      # rounding to x decimals.
pi
exp(1)             # Eulers number
sin(),cos(),tan()  # trigonometric functions
min(), max()       # returns the lowest/highest value of a vector/matrix

```

As a few examples enter the following calculations and make sure to understand the commands:

```

1.8+2
1.8-2
1.8*2
1.8/2
2+2*3
(2+2)*3
2^3
8^(1/3)
3^2
9^0.5
2^2*2+2
2^(2*((0.2+0.3)*(1+2)))+4
sqrt(2)
exp(1)
exp(2)
log(7.389056)
log(exp(3))
log(100,10)
abs(1.8-2)
round(sqrt(2),2)
round(sqrt(2),4)
pi
sin(pi/2)
sin(pi/2)
sin(pi)
tan(pi)
x=2+3i
y=4+1i
x
y
x+y
x*y

```

Attention:

```
1.224606e-16
```

is interpreted as 1.224606×10^{-16} . This is often exactly zero, but for internal

memory purposes (R uses double-precision floating-point numbers and therefore only has up to 16 decimals), can be a rounding mistake.

1.5 Further data objects

Besides classical data objects such as scalars, vectors, and matrices there are three further data objects in R:

1. The **array**: A matrix but with more dimensions. Here is an example of a $2 \times 2 \times 2$ -dimensional array

```
myFirst.Array <- array(c(1:8), dim=c(2,2,2))
```

2. The **list**: In lists you can organize different kinds of data. E.g., consider the following example:

```
myFirst.List <- list("Some_Numbers" = c(66, 76, 55, 12, 4, 66, 8, 99),
                    "Animals"       = c("Rabbit", "Cat", "Elephant"),
                    "My_Series"     = c(30:1))
```

3. The **data frame**: A `data.frame` is a list-object but with some more formal restrictions (e.g., equal number of rows for all columns). As indicated by its name, a `data.frame`-object is designed to store data:

```
myFirst.Dataframe <- data.frame("Credit_Default" = c(0, 0, 1, 0, 1, 1),
                                "Age"            = c(35, 41, 55, 36, 44, 26),
                                "Loan_in_1000_EUR" = c(55, 65, 23, 12, 98, 76))
```

1.6 R help

There exist help files for all commands in R. For example consider the help file of the function `sum()`. You can access the help file with

```
?sum
```

The file usually contains the categories: Description, Usage, Arguments, Details, Value and References.

After the general description follows the syntax of the command. This usually indicates which (optional) arguments the command allows. Then follows `na.rm`, which indicates whether missing values (na) will be removed (rm). The default is `FALSE`.

```
x <- c(1, 2, NA, 4, 5)
x
```

```
## [1] 1 2 NA 4 5
```

```
sum(x)
```

```
## [1] NA
sum(x, na.rm=T)
```

```
## [1] 12
```

Details describes the exact calculation (trivial in this case), and some command specific characteristics. Value describes what is returned, in the case of `sum()` it is obviously a sum of numbers, given that numeric data was used as an input.

1.7 R packages

There are tons of R packages that are useful for different aspects of data analysis. Some packages that can help you make beautiful plots and that you might want check out are the package *ggplot2* and the collection of packages *tidyverse*. You can install the packages with `install.package(packagename)`. In order to use the package type `library(package)`. The package has to be installed only once, but you have to load the package with `library(package)` every time you want to use it.

1.8 Exercises I

Exercise 1:

Try to shorten the notation of the following vectors as much as possible, using : notation:

- `x <- c(157, 158, 159, 160, 161, 162, 163, 164)`
- `x <- c(10, 9, 8, 7, 6, 5, 4, 3, 2, 1)`
- `x <- c(-1071, -1072, -1073, -1074, -1075, -1074, -1073, -1072, -1071)`

Exercise 2:

The `'.'` operator can be used in more complex operations along with arithmetic operators, and variable names. Have a look at the following expressions, and write down what sequence you think they will generate. Then check with R.

- `(10:20) * 2`
- `105:(30 * 3)`
- `10:20*2`
- `1 + 1:10/10`
- `2^(0:5)`

Exercise 3:

R has several functions for rounding. Let's start with `floor`, `ceiling`, and `trunc`:

`floor(x)` rounds to the largest integer not greater than `x` `ceiling(x)` rounds to the smallest integer not less than `x` `trunc(x)` returns the integer part of `x`

Below you will find a series of arguments (x), and results (y), that can be obtained by choosing one or more of the 3 functions above (e.g. `y <- floor(x)`). Which of the above 3 functions could have been used in each case? First, choose your answer without using R, then check with R.

- a. `x <- c(300.99, 1.6, 583, 42.10)`
`y <- c(300, 1, 583, 42)`
- b. `x <- c(152.34, 1940.63, 1.0001, -2.4, sqrt(26))`
`y <- c(152, 1940, 1, -2, 5)`
- c. `x <- -c(3.2, 444.35, 1/9, 100)`
`y <- c(-3, -444, 0, -100)`
- d. `x <- c(35.6, 670, -5.4, 3^3)`
`y <- c(36, 670, -5, 27)`

Exercise 4:

Consider the following vectors:

```
u <- c(5, 6, 7)
```

```
v <- c(2, 3, 4)
```

Perform the following operations:

- Add u and v
- subtract v from u
- multiply u by v element by element
- divide u by v element by element
- calculate the inner product of u and v
- calculate the outer product of u and v
- raise u to the power of v element by element

Then check your answer with R.

Chapter 2

Create and load data sets

We will use R to analyze both simulated and real data. Simulated data will be crucial to study small sample properties of estimators.

2.1 Simulated data

We start by considering data that is simulated by the user. We can create data sets by using functions like `rnorm()`, `rexp()` or `runif()`. After the data is created we can work with it as shown in the following example:

```
n = 100 # number of observations

Z = rnorm(n, mean = 5, sd = 3)      # draw from normal distribution
mean(Z)

## [1] 4.585082
sd(Z)

## [1] 2.992404
E = rexp(n, rate = 3)              # draw data from exponential distribution
mean(E)

## [1] 0.2922543
sd(E)

## [1] 0.3360123
U = runif(n, min = 3, max = 5)     # draw data from uniform distribution
mean(U)

## [1] 3.886656
```

```
sd(U)
```

```
## [1] 0.5311706
```

Analogous functions exist for many other distributions in R, and you can find them by searching on the web.

2.2 Load datasets

It is also possible to work with observed data, which will be loaded into R. Here, we will use some real data, namely a movie database from IMDB that you can download [here](#). The data set contains, among others, the budget, the revenue, and the average rating of movies.

Once the data is saved in your working directory, you can load it using

```
data_set = read.csv("tmdb_5000_movies.csv")
```

which stores the data in a data frame. You can use the `View()` function to look the data.

After the file is loaded, we can work with the data using `$` to access certain parts of the data set such as

```
data_set$original_title[1:5]
```

```
## [1] "Avatar"
```

```
"Pirates of the Caribbean: At World's End"
```

```
## [5] "John Carter"
```

```
data_set$budget[1:5]
```

```
## [1] 237000000 300000000 245000000 250000000 260000000
```

```
mean(data_set$budget[1:5])
```

```
## [1] 258400000
```

We will return to this data set when we talk about regression analysis later.

Apart from `read.csv` there exist also `read.dta`, `read.xls` and many more to load suitable data sets.

Another important function is `readRDS` that is used to read in a data set that was saved using the function `saveRDS`. These functions can be useful in case properties of your R object are not preserved when saved in other formats, such as a csv file.

2.3 Save datasets

There are also multiple ways to save your own data. While `read.csv` can be used to read in data from a csv file, its counterpart `write.csv` can be used to save a suitable object as a csv file. The most important arguments for this function are `x`, the object you want to save, and `file`, which provides the path where to save the file.

```
write.csv(data_set, <my_path>)
```

Similarly the function `saveRDS` allows you to save a generic R object to storage. One advantage of this function compared to `write.csv` is that it preserves properties of your object that might be lost when saved as a csv file. For example, conversions from factors to strings might occur when saving a `data.frame` as a csv file.

The package `readxl` and `openxlsx` from the tidyverse family can be used to read in and write to excel files, respectively. This format is usually not the best option to store your data.

2.4 Exercises II

Exercise 1:

Simulate a vector containing 10000 independent normally distributed random variables with means of 1 and variances of 2. How many of them are negative? What is the expected fraction of negative values?

Exercise 2:

Simulate `u` and `v` as independent log-normally distributed random vectors of size 200 each. Pick means and variances you like. Then let `w` be equal to `u+v`. Now calculate the following:

- the sample means of `u`, `v`, and `w`
- the sample variances of `u`, `v`, and `w`
- all pairwise covariances

Exercise 3:

Take the objects you created in the previous exercises from this chapter and save them to storage in your preferred format. Then close your session, clean your workspace and read them in again. This might be tedious at first, but integrating the export of data into your workflow might save you from time costly mistakes in the future.

Chapter 3

Simple programming

We often want to repeat specific operations several times. For example, we might want to repeat an operation for different elements of a given data structure, such as columns of a matrix, or perform an operation until a satisfactory result is reached, such as running an algorithm until the value converges to a prespecified level. In addition, we often need to check whether certain conditions are fulfilled, such as whether elements of a vector are equal to 0, and perform an operation depending on the result.

3.1 The for-loop

A for-loop is a procedure in which a particular routine is repeated and only an index variable is changed.

```
x=0
for (i in 1:3) {
  x=x+i
  print(x)
}
```

```
## [1] 1
## [1] 3
## [1] 6
```

We can also use it to study the behavior of the arithmetic mean

```
out = rep(0,100)
n = 20
for (i in 1:100) {
  x = rnorm(n, mean = 0, sd = 1)
  out[i]=mean(x)
```

```

}
mean(out)

## [1] 0.02992899
sd(out)

## [1] 0.2312502
out = rep(0,100)
n = 200
for (i in 1:100) {
  x = rnorm(n,mean = 0,sd = 1)
  out[i]=mean(x)
}
mean(out)

## [1] 0.008600372
sd(out)

## [1] 0.06805146

```

3.2 The if-condition

Within a for-loop, it may be useful to perform arithmetic operations only under certain conditions. The basic syntax for this is

```

if (condition){
  statement
}
else{
  alternative
}

```

where else can remain unspecified.

```

###If-condition###
y <- c(1,4,5,2,7,8,2,4)
N <- length(y)
1:N
##
y.sq <- numeric(N)#creates an empty numeric vector of length N

for(i in 1:N){ #initializing the for-loop
  y.sq[i] <- y[i]^2#every element of y.sq is replaced by the respective squared value.
  if(i == N){#only when this condition is met, i.e. in the last iteration of the loop
    print(y.sq)#is the vector printed
  }
}

```

```
}  
}
```

In shorter form the `ifelse()` command is an alternative.

```
ifelse(test, yes, no)
```

3.3 The while-loop

The while-loop performs a program (“statement”) as long as certain data conditions (“test expression”) are met. The basic syntax is

```
while (testexpression)  
{  
  #increment (i+1);  
  statement  
}
```

Here, `test_expression` is evaluated and the body of the loop is entered if the result is TRUE.

```
i <- 1  
  
while (i < 6) {  
  print(i)  
  i = i+1  
}
```

```
## [1] 1  
## [1] 2  
## [1] 3  
## [1] 4  
## [1] 5
```

3.4 Time saving and loops

One easy way to make your loops in R faster is to preallocate space for your results instead of appending them to a vector. This is caused by the underlying architectural choice of R that is called copy-on-modify. In some cases it leads to your whole data being copied from one place in memory to another, when you simply wanted to append another value to your vector.

If you look at the following code fragments, the second one is about 70 to 80 times faster!

```
# The slow but simple way  
time <- system.time(  

```

```

{results <- c() # empty vector
for(i in 1:100000){
  results <- c(results, rnorm(1))
}}
)
print(time)

```

```

##      user  system elapsed
##    9.50    0.47    9.97

```

```

# The right way to do it
time2 <- system.time(
  {results2 <- vector(mode = "double", length = 100000)
  for(i in 1:100000){
    results2[i] <- rnorm(1)
  }}
)
print(time2)

```

```

##      user  system elapsed
##    0.06    0.00    0.06

```

If you can avoid a loop altogether and use vector operations instead, your code will be even faster.

```

# The right way to do it
time3 <- system.time(
  {results3 <- rnorm(100000)
  }
)
print(time3)

```

```

##      user  system elapsed
##    0.00    0.00    0.02

```

Try to keep this in mind when designing your own code. In R this is one of the most frequent and easily avoidable reasons for why code is slow.

3.5 Exercises III

Exercise 1:

Write a for-loop that iterates over the numbers 6 to 14 and prints the cube of each number using `print()`

Exercise 2:

Write a while-loop that prints out standard random normal numbers (use `rnorm()`) but stops (breaks) if you get a number bigger than 1.

Exercise 3:

Using a for-loop simulate the flip a coin twenty times, keeping track of the individual outcomes (1 = heads, 0 = tails) in a vector that you preallocated.

Chapter 4

Functions

If you want to make your code more readable and easily understandable, you should split up complicated tasks into more simple ones. A convenient way to do this is defining a function that generalizes a step of your code. If you define this function in advance, you can call it later to perform the task, which might make the code more easily readable and enables you to use paradigmas such as recursion or metaprogramming.

4.1 Function definition

A function definition follows a specific structure.

```
my_function <- function(argument1, argument2){  
  # here I describe what I want to do with my arguments  
  # For example multiply them, assuming that they are numerical.  
  
  my_return_value = argument1 * argument2  
  
  # and using return() we tell our function what to return  
  return(my_return_value)  
}  
  
my_function(argument1 = 5, argument2 = 7)
```

```
## [1] 35
```

The function can also be called without explicitly naming the arguments, but if you don't know your code very well, this might lead to difficulties when somebody else or your future self might try to understand the code.

```
my_function(2,5)
```

```
## [1] 10
```

Another useful feature is the definition of default values for your arguments such that they can be omitted when calling a function.

```
my_2nd_function <- function(argument1, argument2, greetings = FALSE){
  my_return_value <- argument1 * argument2
  if(greetings == TRUE){
    print("Hello user, the cake is a lie.")
  }
  return(my_return_value)
}
```

```
my_2nd_function(2, 7)
```

```
## [1] 14
```

```
my_2nd_function(3, 12, TRUE)
```

```
## [1] "Hello user, the cake is a lie."
```

```
## [1] 36
```

4.2 Recursion

In some settings using a recursive approach instead of loops can create more readable code. In simple terms, recursion in programming is calling a function in its own body. A simple example - even though very inefficient from a computational standpoint - is calculating factorials.

```
factorial <- function(n){
  if(n == 1) return(1)
  else return(n*factorial(n-1))
}
```

```
factorial(17)
```

```
## [1] 3.556874e+14
```

Often recursion relies on a principle called “divide and conquer”. We test for a simple base case, like n being equal to one in the case of factorials, and otherwise reducing the problem to a more simple one.

Another example is the calculation of Fibonacci numbers using recursion.

```
fibonacci <- function(n){
  if(n == 1 || n == 2) return(1)
```

```
    else return(fibonacci(n-1) + fibonacci(n-2))
}

fibonacci(20)

## [1] 6765
```

4.3 Exercises IV

Exercise 1:

Write a function that takes 2 integers as inputs and outputs whether or not the first integer is divisible by the second as a boolean.

Exercise 2:

Try to solve the problem of adding all squares from 1 to n first using loops and then recursion. Think about the underlying way in which the function works. Which approach do you prefer? Why?

Exercise 3:

Write a function that receives a single vector of real numbers as its argument and returns an ordered vector with the same entries.

If you are interested to see recursion in action look for mergesort, that solves this problem with a cool recursive approach.

Chapter 5

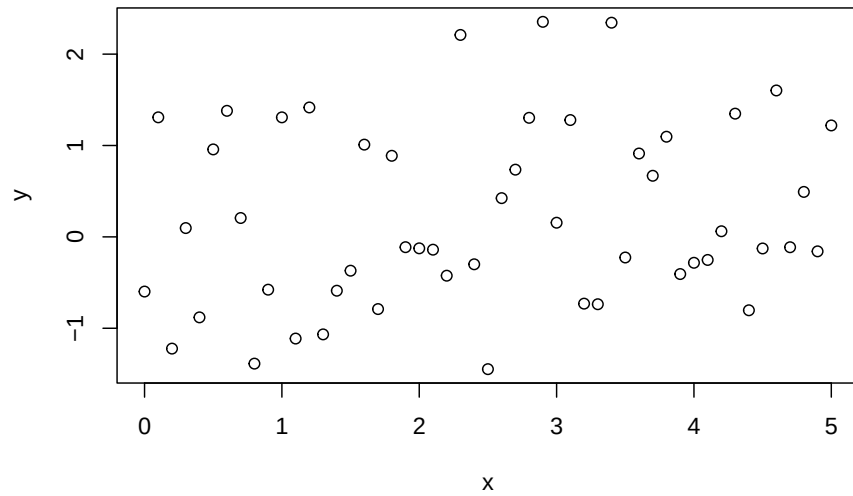
Plotting in R

It is often useful to visualize data by using plots. R offers many different possibilities to create plots, ranging from simple scatterplots to animated plots, that you can export as gifs or even interactive webapps. The built-in capabilities of R offer many tools, to quickly generate plots and packages like ggplot2 from the tidyverse family offer a full plotting experience built on the so called “Grammar of Graphics”.

5.1 The plot-function

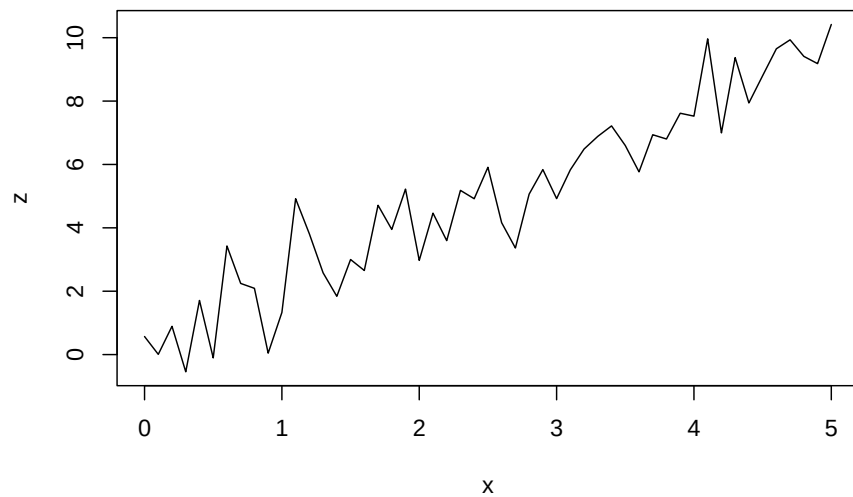
In its simplest form, the plot function in R delivers a scatterplot. In the call below, `type = "p"` could be omitted as it is the default value if the argument is not provided in the call.

```
x <- seq(0, 5, 0.1)
y <- rnorm(n = length(x))
plot(x = x, y = y, type = "p")
```

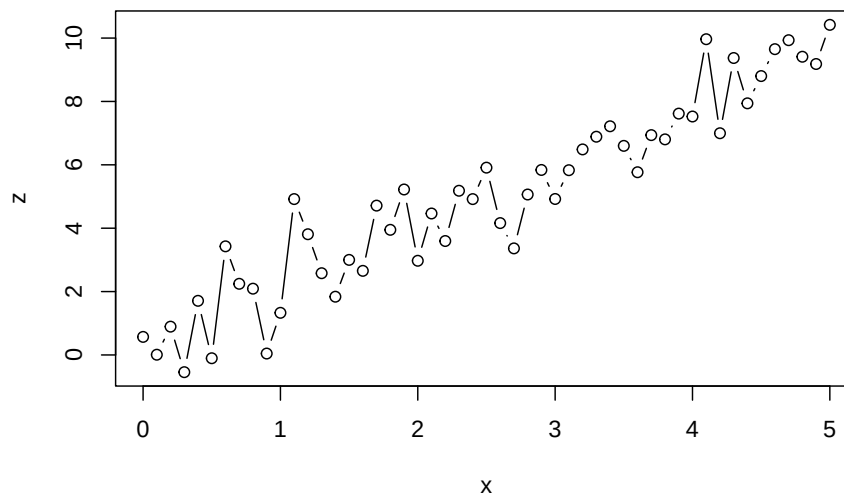


If we instead want a simple line plot or a combination of both, we can specify this behavior using the `type` argument.

```
z <- 2*x + rnorm(n = 51)
plot(x = x, y = z, type = "l")
```



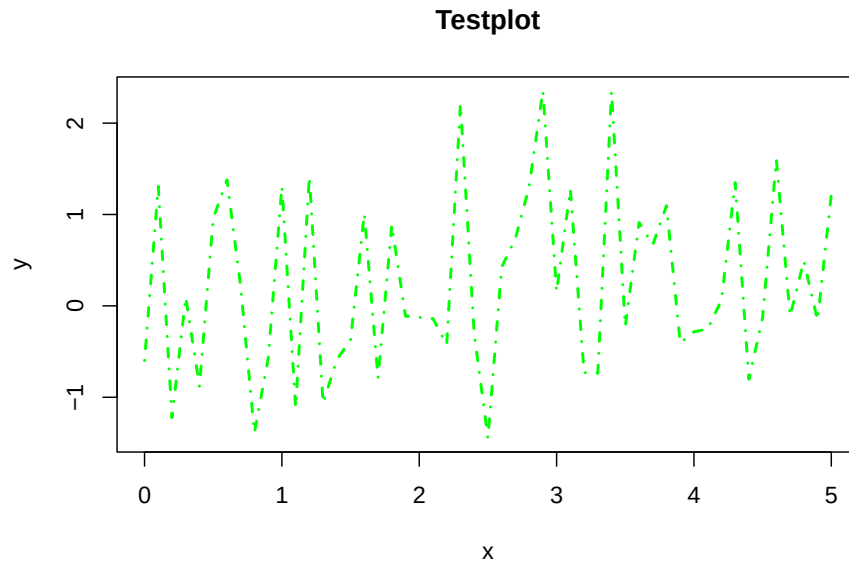
```
plot(x = x, y = z, type = "b")
```



Other arguments to the plot function can be used to specify color, linetype (dashed etc.), linewidth, title, subtitle and many more. The help files `?plot` and `?par` can be very helpful when trying to achieve a specific look for your plot.

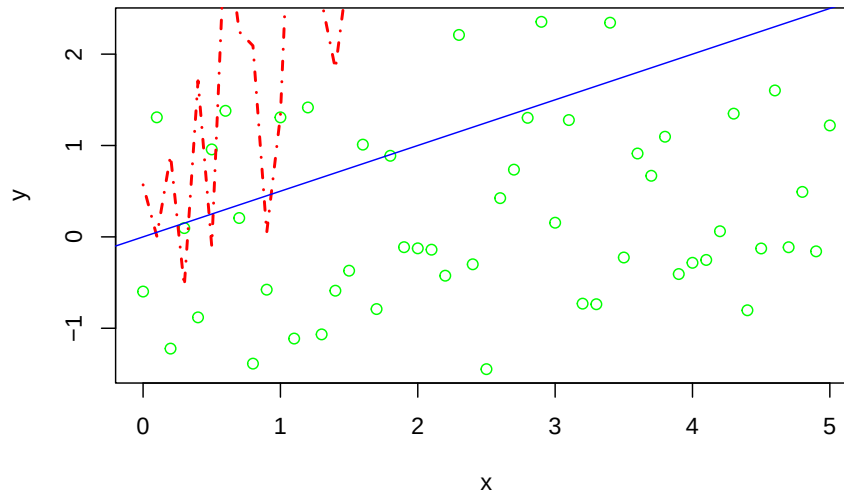
An example could look a little like this:

```
plot(x = x, y = y, type = "l", lty = 4, lwd = 2, col = "green", main = "Testplot")
```



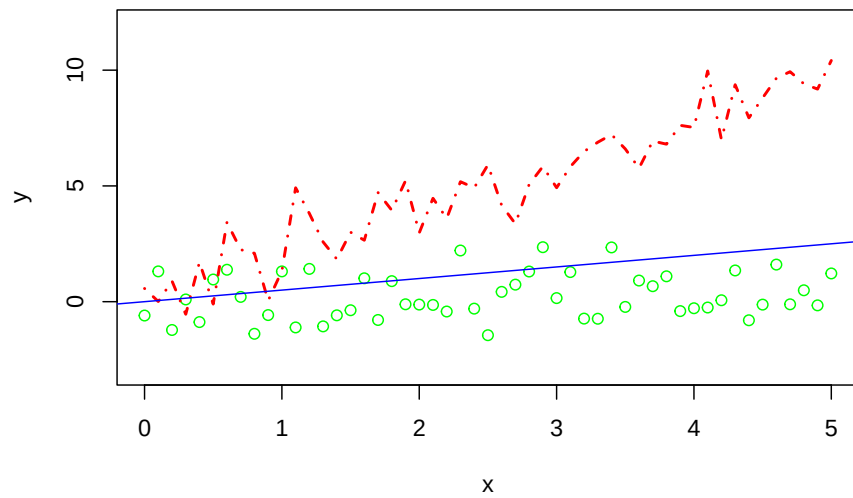
If we want to show multiple things in one diagram it can be useful to plot multiple objects in one coordinate grid. R provides some ways to do so, the most simple ones being the `lines` function that works similar to the `plot` function and the `abline` function which can be used to add straight lines.

```
plot(x = x, y = y, type = "p", col = "green")
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
abline(a=0, b=0.5, col = "blue")
```



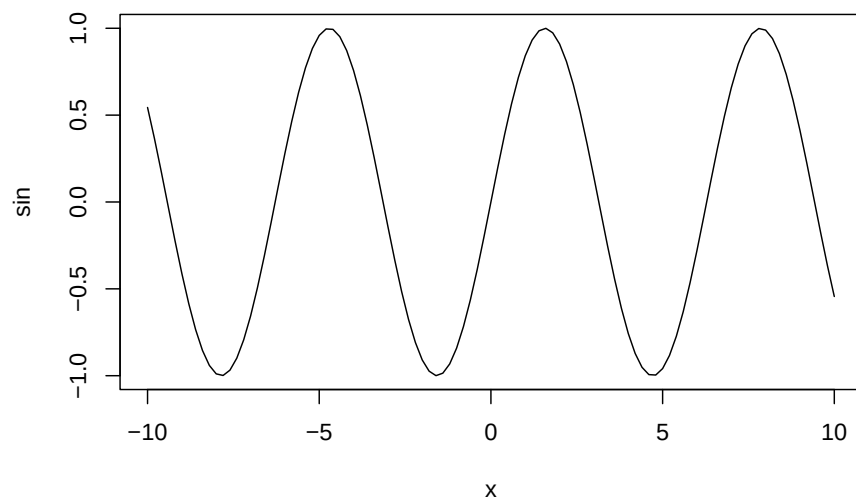
As you may see from this plot, our y-axis is not adjusted to our newly added line. But we have a possibility to manually adjust the range of our axes if we choose to do so. This is done using the `xlim` and `ylim` arguments in the original `plot` function.

```
plot(x = x, y = y, type = "p", col = "green", xlim = c(0,5), ylim = c(-3, 12))
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
abline(a=0, b=0.5, col = "blue")
```

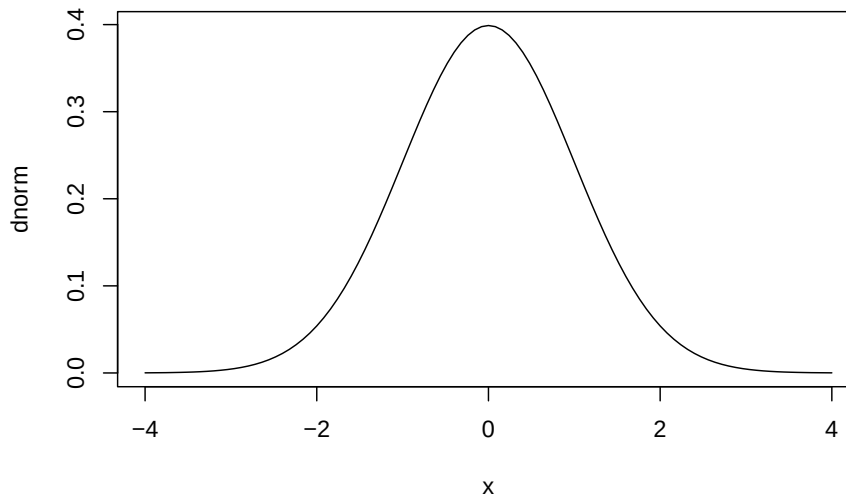


It is also possible to plot functions like $\sin(x)$ or the standard normal density.

```
plot(sin, xlim = c(-10, 10))
```



```
plot(dnorm, xlim = c(-4, 4))
```



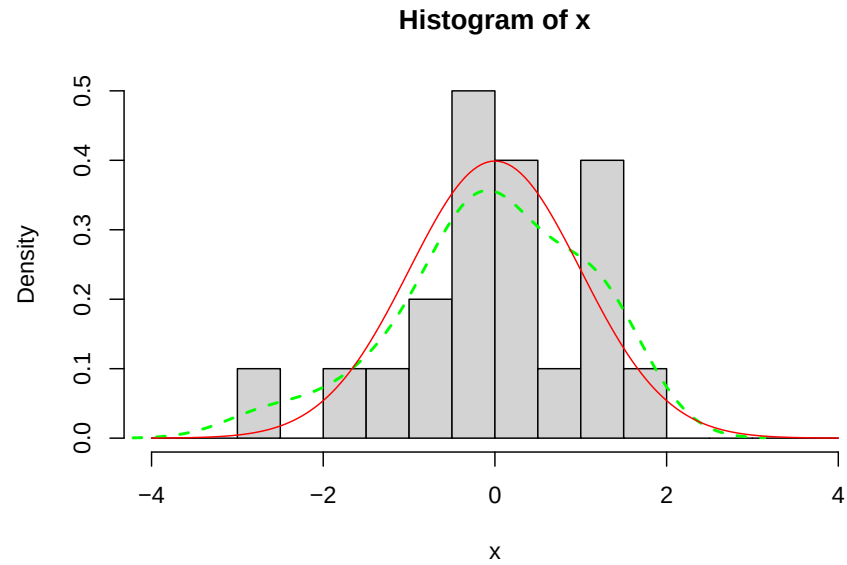
There are many other useful plotting functions that are integrated in base R. We won't cover all of them here, but instead focus on some of the most frequently needed.

5.2 Histograms and kernel densities

Other useful plotting functions that are offered in the default R experience include histograms with the function `hist`. With a combination of `plot` and `density` we get access to kernel density plots. A kernel density estimator is a nonparametric estimator of density of random variable based on a sample. Details about the estimator are beyond the scope of these notes. In the settings where we will use them, the default choices of the estimator will yield accurate results. Combining our previous knowledge with these possibilities, we can create plots like the following (note: in `hist` the argument `freq = FALSE` leads to relative frequency instead of counts. See `?hist` for more details.):

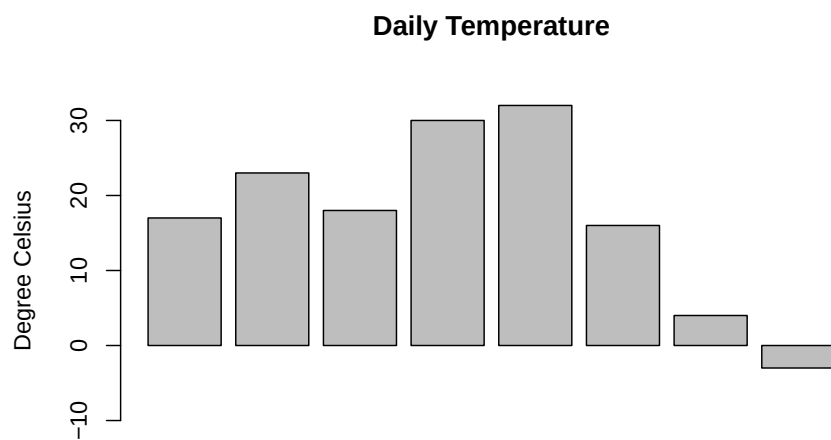
```
x <- rnorm(n = 20)
my_dens <- density(x)

hist(x, freq = FALSE, breaks = seq(-4, 4, 0.5), xlim = c(-4, 4))
lines(my_dens, col = "green", lty = 2, lwd = 2)
lines(x = seq(-4, 4, 0.05), y = dnorm(seq(-4, 4, 0.05)), col = "red", lty = 1)
```



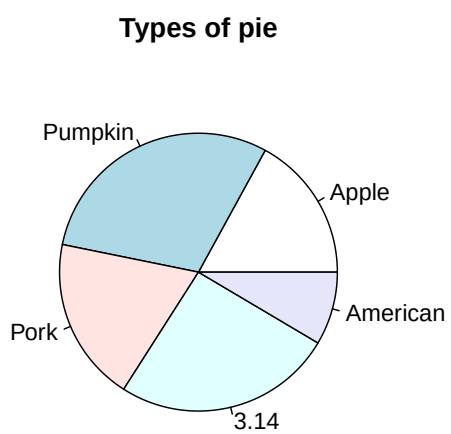
Further functions include barplots:

```
temperatures <- c(17,23,18,30,32,16,4,-3)
barplot(temperatures,
        ylim = c(-15, 35),
        main = "Daily Temperature",
        ylab = "Degree Celsius")
```



and piecharts:

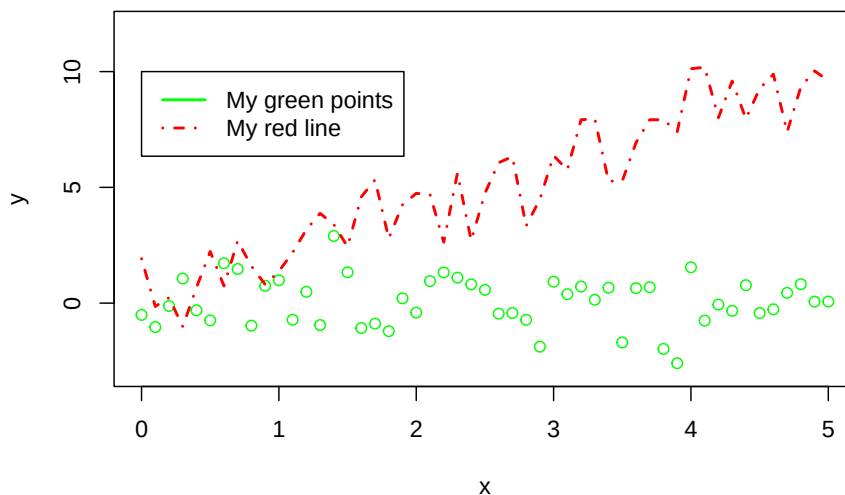
```
slices <- c(8, 14, 9, 12, 4)
labels <- c("Apple", "Pumpkin", "Pork", "3.14", "American")
pie(slices, labels = labels, main = "Types of pie")
```



Another important part of plots - like comments in code - is a legible legend. R has built-in support for adding a legend to your plots. Going back to our combined plot from above, one can add a legend as shown in the following example. Here the `x` and `y` arguments specify the position of the legend in the plot and the other arguments specify its contents. Since a legend has to be customized for each plot, one should have a look in the documentation `?legend`.

```
x <- seq(0, 5, 0.1)
y <- rnorm(n = length(x))
z <- 2*x + rnorm(n = 51)

plot(x = x, y = y, type = "p", col = "green", xlim = c(0,5), ylim = c(-3, 12))
lines(x = x, y = z, type = "l", lwd = 2, lty = 4, col = "red")
legend(x = 0, y = 10,
      legend = c("My green points", "My red line"),
      col = c("green", "red"),
      lty = c(1, 4),
      lwd = 2)
```



If one decides to pursue a tidyverse approach, the package `ggplot` allows for beautiful, heavily customized plots. An example can be seen below:

```
library(ggplot2)
library(ggforce)
library(hrbrthemes)
```

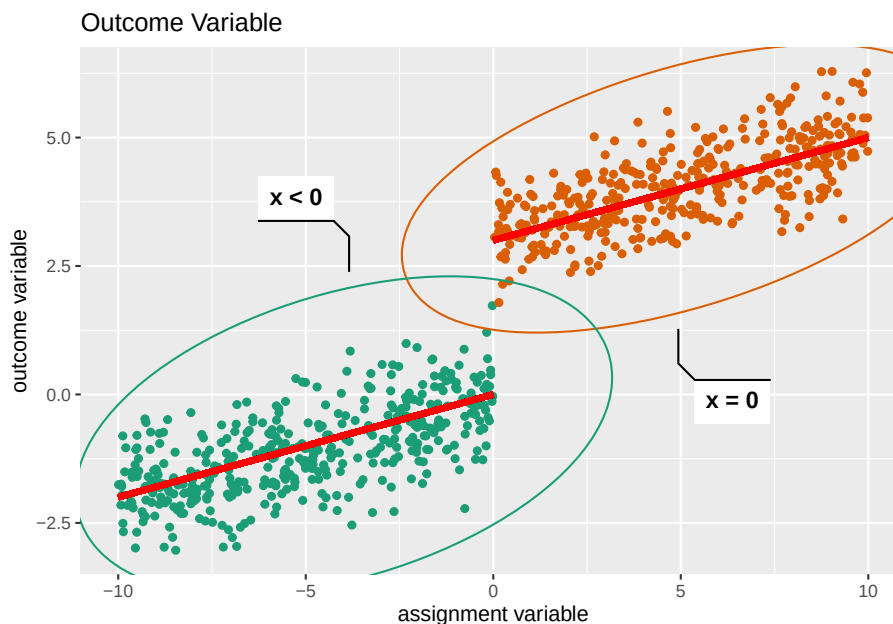
```

n = 800
x <- runif(n, min = -10, max = 10)
treatment <- ifelse(x>0,1,0)
y <- 0.2*x + treatment*3 + rnorm(n, mean = 0, sd = 0.6)
visualise.data <- data.frame(x, treatment, y)

p1 <- ggplot(visualise.data, aes(x=x, y=y, colour = treatment == 1)) +
  labs(title = "Outcome Variable", y = "outcome variable", x = "assignment variable") +
  geom_point() +
  geom_mark_ellipse(aes(filter = treatment == 1,
                        label = 'x \U002265 0')) +
  geom_mark_ellipse(aes(filter = treatment == 0,
                        label = 'x < 0')) +
  geom_segment(aes(x = -10, y = -2, xend = 0, yend = 0), color = 'red', size = 1.5) +
  geom_segment(aes(x = 0, y = 3, xend = 10, yend = 5), color = 'red', size = 1.5) +
  #theme_ipsum(axis_title_size = 18) +
  theme(legend.position = "none",
        legend.title = element_text(size = 18)) +
  scale_colour_brewer(type = 'qual', palette = 'Dark2')

p1

```



The possibilities of ggplot2 are vast and can be intimidating, but it offers a consistent plotting experience integrated into a tidy data workflow using the tidyverse family of packages.

5.3 Exercises V

Exercise 1:

Draw samples of size 1000 from exponential distributions with varying values of the parameter. Use histograms to visualize differences in the distributions.

Exercise 2:

Take the `mtcars` dataset that is included in base R and plot `mpg` (miles per gallon) against `wt` (weight) as a scatterplot. Add a red line with `abline` by eyeballing the parameters that could come out of a regression.

Exercise 3:

Write a function that takes as its arguments a sample size `n` and a boolean that indicates, if plotting should take place. Let this function return a sample of size `n` drawn from a standard normal. If the plotting argument was true, let it also plot your sample in a way that seems sensible to you.

Chapter 6

Monte Carlo simulations

6.1 Some important concepts

We will spend a lot of time learning about statistical properties of estimators. To be precise, let $\hat{\theta}_n$ be an estimator of a parameter θ . The estimator is a function of the sample, say $\{X_1, X_2, \dots, X_n\}$. The sample consists of random variables and any function of the sample, including $\hat{\theta}_n$, is a random variable as well. That is, for different realizations of the data, we get different realizations of the estimator.

These arguments can be confusing because in practice we typically only have one data set and therefore only one realization of the estimator. But we could also think of all the other realizations of the data we could have obtained. For example, consider your starting salary after graduation. Let θ be the expected starting salary, which is a fixed number that we do not know. After everyone started working and we see your actual starting salaries, we can calculate the sample average as an estimate of the expected value, but this estimate is just one potential realization. If things had turned out slightly different, your starting salaries might have been different and we would have seen a different realization of the estimator.

Since an estimator is a random variable, it has a distribution and (typically) a mean and a variance. We call an estimator unbiased if

$$E(\hat{\theta}_n) = \theta$$

That is, the average value of the estimator over all the hypothetical data sets we could have obtained is equal to θ . Unbiasedness can hold for any sample size n and is therefore called a small-sample property. Unbiasedness does not imply that $\hat{\theta}_n$ is necessarily close to θ .

We will define what it means for an estimator to be consistent:

$$\hat{\theta}_n \rightarrow_P \theta$$

and asymptotically normally distributed:

$$\sqrt{n}(\hat{\theta}_n - \theta) \rightarrow_D N(0, V_\theta^2)$$

These properties only hold in the limit and are called large-sample properties. Knowing the large-sample distribution is particularly useful because it allows us to test hypotheses and obtain confidence intervals. For example, let $\hat{V}_\theta$ be a consistent estimator of V_θ . Then

$$P\left(\theta \in \left[\hat{\theta}_n - 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}, \hat{\theta}_n + 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}\right]\right) \rightarrow 0.95$$

and the interval

$$\left[\hat{\theta}_n - 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}, \hat{\theta}_n + 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}\right]$$

is a 95% confidence interval for θ . That is, looking at all hypothetical data sets, θ is in 95% of the confidence intervals. Notice that in different data sets, we get different intervals, but the parameter θ does not change.

The 95% coverage probability only holds in the limit, but for any fixed sample size

$$P\left(\theta \in \left[\hat{\theta}_n - 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}, \hat{\theta}_n + 1.96 \frac{\hat{V}_\theta}{\sqrt{n}}\right]\right)$$

could be much smaller than 95%. This actual coverage probability is unknown in applications.

The goal of Monte Carlo simulations is typically to investigate small sample properties of estimators, such as the actual coverage probability of confidence intervals for fixed n . To do so, we can simulate many random samples from an underlying distribution and obtain the realization of the estimator for each sample. We can then use the realizations to approximate the actual small sample distribution of the estimator and check properties, such as coverage probabilities of confidence intervals. Of course, the properties of the estimator depend on which underlying distribution we sample from.

6.2 Simple simulation example

As a simple example, let $\{X_1, X_2, \dots, X_n\}$ be a random sample from some distribution F_X . Define $\theta = E(X)$ and let

$$\hat{\theta}_n = \bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$$

and

$$s_n = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X}_n)^2}$$

Applications of a Law of Large Numbers and a Central Limit Theorem for iid data imply that

$$\hat{\theta}_n \rightarrow_P \theta,$$

$$s_n \rightarrow_P sd(X),$$

and

$$\sqrt{n}(\hat{\theta}_n - \theta) \rightarrow_D N(0, Var(X))$$

The previously discussed confidence interval for $E(X)$ is

$$\left[\bar{X}_n - 1.96 \frac{s_n}{\sqrt{n}}, \bar{X}_n + 1.96 \frac{s_n}{\sqrt{n}} \right]$$

6.3 Consistency

The estimator $\hat{\theta}_n$ is weakly consistent if for any $\varepsilon > 0$

$$P\left(\left|\hat{\theta}_n - \theta\right| > \varepsilon\right) \rightarrow 0$$

or

$$P\left(\left|\hat{\theta}_n - \theta\right| \leq \varepsilon\right) \rightarrow 1$$

That is, as long as n is large enough, $\hat{\theta}_n$ is in an arbitrarily small neighborhood around θ with arbitrarily large probability.

To illustrate this property, let's return to the example with sample averages and expectations. Let $\{X_1, X_2, \dots, X_n\}$ be a random sample from the Bernoulli distribution with $p = P(X_i = 1) = 0.7$. Let $\theta = E(X) = p$ and let $\hat{\theta}_n$ be the sample average. We start with a sample size of 20 and set the parameters:

```
n <- 20
p <- 0.7
theta = p
```

We now want to take S random samples and obtain S corresponding sample averages, where S is ideally a large number. The larger S , the more precise our results will be. It is also useful to set a seed.

```
set.seed(4519)
S <- 10000
X <- replicate(S, rbinom(n, 1, p))
theta_hat <- colMeans(X)
```

Now for some $\varepsilon > 0$, we can check $P\left(\left|\hat{\theta}_n - \theta\right| \leq \varepsilon\right)$:

```
eps <- 0.1
mean(abs(theta_hat - p) <= eps)
```

```
## [1] 0.647
```

We can also calculate this probability for different values of $\varepsilon > 0$. I use a loop here for readability, but you can try using matrices on your own. Since I want to repeat the following steps for different sample sizes, it is useful to first define a functions that performs all operations:

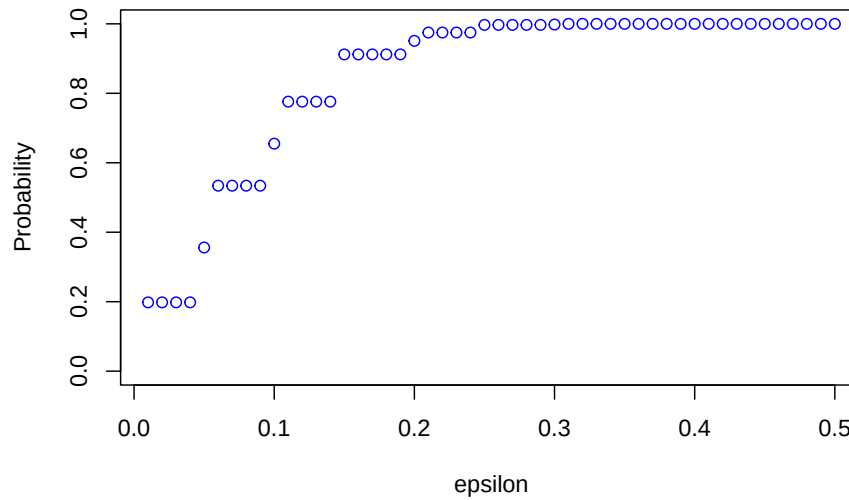
```
calc_probs_consistency <- function(n,S,p,all_eps) {
  n_eps = length(all_eps)
  X <- replicate(S,rbinom(n,1,p))
  theta_hat <- colMeans(X)
  probs = rep(0, n_eps)
  for (i in 1:n_eps) {
    probs[i] = mean(abs(theta_hat - p) <= all_eps[i])
  }
  return(probs)
}
```

We can now call the function for any n , S , p , and a vector containing different values for ε . For each of these values, the function then returns a probability.

```
all_eps = seq(0.01, 0.5, by= 0.01)
probs <- calc_probs_consistency(n = 20, S = 1000, p = 0.7, all_eps = all_eps)
```

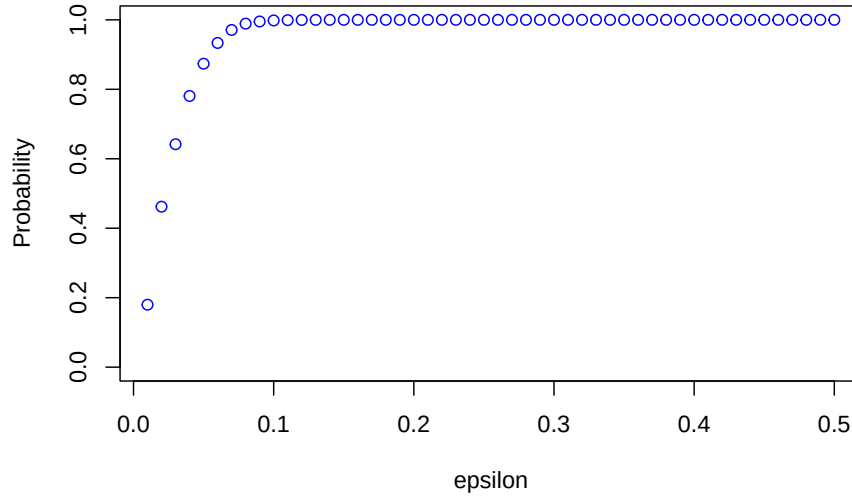
Next, we can plot all probabilities:

```
plot(all_eps,probs,ylim = c(0,1),xlab="epsilon",ylab="Probability", col = 'blue')
```



We can see from the plot that $P\left(\left|\hat{\theta}_n - \theta\right| \leq \varepsilon\right)$ is quite small for small values of ε . Consistency means that for any $\varepsilon > 0$, the probability approaches 1 as $n \rightarrow \infty$. We can now easily change the sample size and see how the results change. For example, take $n = 200$:

```
probs <- calc_probs_consistency(n = 200, S = 10000, p = 0.7, all_eps = seq(0.01, 0.5, by= 0.01))
plot(all_eps,probs,ylim = c(0,1),xlab="epsilon",ylab="Probability", col = 'blue')
```



6.4 Asymptotic normality

We know from the Central Limit Theorem that

$$\frac{\sqrt{n}(\bar{X}_n - E(X))}{s_n} \rightarrow_D N(0, 1)$$

where s_n is the sample standard deviation. Convergence in distribution means that the distribution of the normalized sample average converges to the distribution function of a standard normal random variable.

Again, we can use the previous simulation setup to illustrate this property. To do so, we calculate

$$P\left(\frac{\sqrt{n}(\bar{X}_n - E(X))}{s_n} \leq z\right)$$

and compare it to the standard normal cdf evaluated at z . We then plot the results as a function of z . Just as before, we define a function that calculates the probabilities:

```
calc_probs_asymp_normal <- function(n,S,p,z) {
  n_z = length(z)
  X <- replicate(S, rbinom(n,1,p))
  theta_hat <- colMeans(X)
  stds <- apply(X, 2, sd)
  probs = rep(0, n_z)
```

```

for (i in 1:n_z) {
  probs[i] = mean( sqrt(n)*(theta_hat - p)/stds <= z[i])
}
return(probs)
}

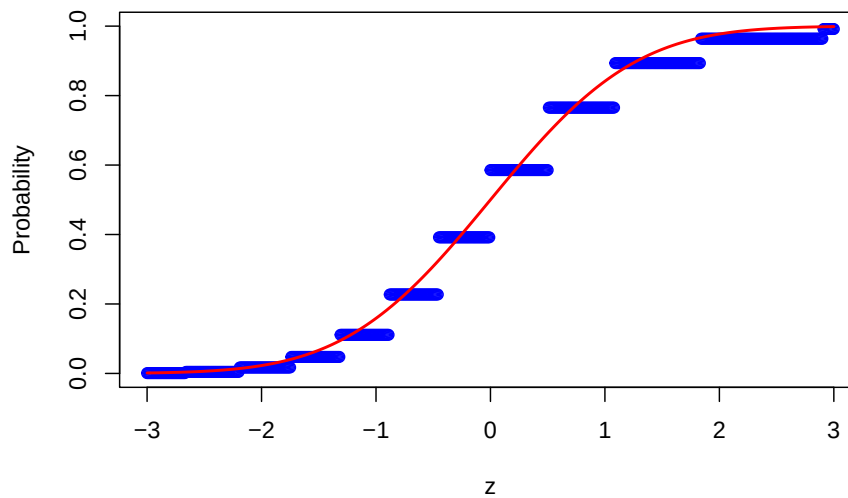
```

We can now plot the results for different sample sizes such as

```

z = seq(-3,3,by=0.01)
probs <- calc_probs_asymp_normal(n = 20, S = 10000, p = 0.7, z = z)
plot(z,probs,ylim = c(0,1),xlab="z",ylab="Probability", col = 'blue')
lines(z, pnorm(z, 0, 1), col = 'red',lwd = 2)

```

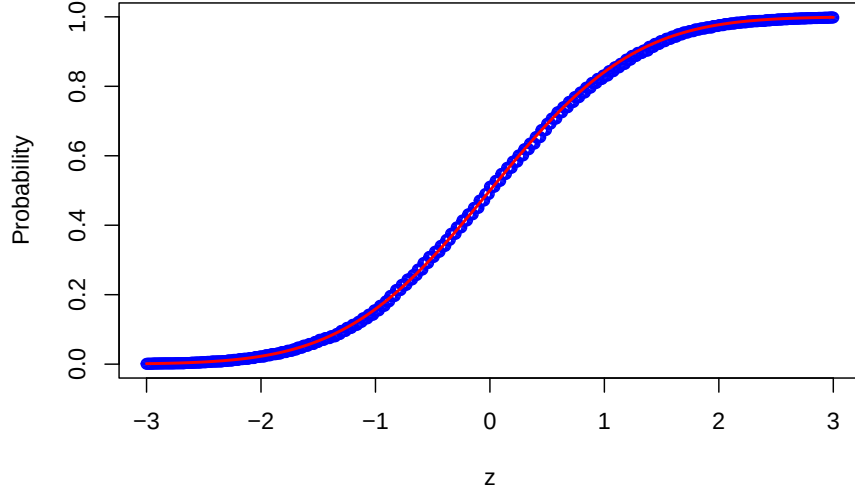


or

```

probs <- calc_probs_asymp_normal(n = 2000, S = 10000, p = 0.7, z = z)
plot(z,probs,ylim = c(0,1),xlab="z",ylab="Probability", col = 'blue')
lines(z, pnorm(z, 0, 1), col = 'red',lwd = 2)

```



6.5 Testing

Let $\{X_1, X_2, \dots, X_n\}$ be a random sample and define $\mu = E(X)$. Suppose we want to test $H_0 : \mu = \mu_0$ where μ_0 is a fixed number. To do so, define

$$t = \frac{\sqrt{n}(\bar{X}_n - \mu_0)}{s_n}$$

An immediate consequence of the Central Limit Theorem is that if H_0 is true then

$$t = \frac{\sqrt{n}(\bar{X}_n - \mu_0)}{s_n} = \frac{\sqrt{n}(\bar{X}_n - \mu)}{s_n} \rightarrow_D N(0, 1)$$

and therefore

$$P(|t| > 1.96 \mid \mu = \mu_0) \rightarrow 0.05$$

Hence, if we reject when $|t| > 1.96$, we only reject a true null hypothesis with probability approaching 5%.

If the null hypothesis is false, we can write

$$t = \frac{\sqrt{n}(\bar{X}_n - \mu_0)}{s_n} = \frac{\sqrt{n}(\bar{X}_n - \mu)}{s_n} + \frac{\sqrt{n}(\mu - \mu_0)}{s_n} \rightarrow_D N(0, 1)$$

Since in this case $\sqrt{n}|\mu - \mu_0| \rightarrow \infty$, it follows that

$$P(|t| > 1.96 \mid \mu \neq \mu_0) \rightarrow 1$$

That is, we reject a false null hypothesis with probability approaching 1. However, for a small sample size, the rejection probability is typically much smaller when μ_0 is close to μ .

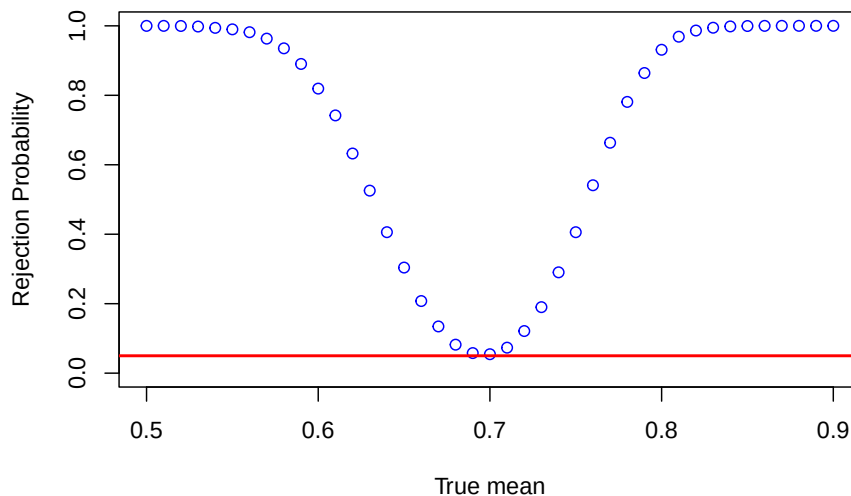
We now want to calculate the small sample rejection probabilities using Monte Carlo simulations. We could either vary μ_0 (the value we test) or μ (the true mean). We take the latter because we can then plot the power functions you have seen in the theoretical part of the class to illustrate the results.

We work with the Bernoulli distribution again and define a function that calculates certain rejection probabilities.

```
calc_rej_probs <- function(n,S,mus,mu0,alpha) {
  n_mus = length(mus)
  probs = rep(0, n_mus)
  for (i in 1:n_mus) {
    X <- replicate(S,rbinom(n,1,mus[i]))
    theta_hat <- colMeans(X)
    stds <- apply(X, 2, sd)
    probs[i] = mean(abs( sqrt(n)*(theta_hat - mu0)/stds ) > qnorm(1-alpha/2) )
  }
  return(probs)
}
```

We can now call the function for different values of μ and plot the rejection probabilities

```
mus = seq(0.5,0.9,by=0.01)
probs <- calc_rej_probs(n = 200, S = 10000, mus = mus, mu0 = 0.7, alpha = 0.05)
plot(mus,probs,ylim = c(0,1),xlab="True mean",ylab="Rejection Probability", col = 'blue')
abline(h=0.05, col = 'red',lwd = 2)
```



Think about how the figure changes as the sample size increases.

6.6 Exercises VI

Exercise 1:

Suppose $X \sim U[0, 2]$ where $U[0, 2]$ denotes the uniform distribution on the interval $[0, 2]$. It is easy to show (make sure to verify this!) that $E(X) = 1$. Now repeat the following two steps at least 1,000 times:

1. Take a random sample from the distribution of X with sample size $n = 20$.
2. Calculate the sample average and a 95% confidence interval for $E(X)$ using the sample from (1).

You should now have 1,000 sample averages and 1,000 confidence intervals, which you can use to answer the following questions:

- a. What is average of your 1,000 sample averages?
- b. What is the standard deviation of your 1,000 sample averages?
- c. How many times is $E(X)$ in the 95% confidence interval?

Exercise 2:

In this exercise, we use the Monte-Carlo method to approximate the number pi. To do so, randomly choose n points from a square and count how many of those

draws lie inside the square's incircle. Based on this results and the formula for the area of a circle, calculate an approximation of π .

Create a plot that portrays your findings in an intuitive manner, such as histograms, density plots, boxplots, etc.) $\rightarrow$

Chapter 7

Regression analysis

7.1 Basic linear model

You have probably all learnt about the linear regression model in a previous econometrics class where

$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_k X_k + \beta_{k+1} + e$$

and

$$E(e \mid X_1, X_2, \dots, X_k) = 0.$$

That is, the conditional mean of the outcome variable Y is a linear function of the regressors $X_1, X_2, \dots, X_k$.

When we have a random sample of size n , we can also write

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + \dots + \beta_k X_{ik} + \beta_{k+1} + e_i, \quad i = 1, 2, \dots, n.$$

Our goal is to estimate the coefficients $\beta_1, \dots, \beta_{k+1}$ using the data $\{Y_i, X_{i1}, X_{i2}, \dots, X_{ik}\}_{i=1}^n$.

7.2 Empirical example

We will now see how to run regressions in R using the movie data set. Again, you can load it using

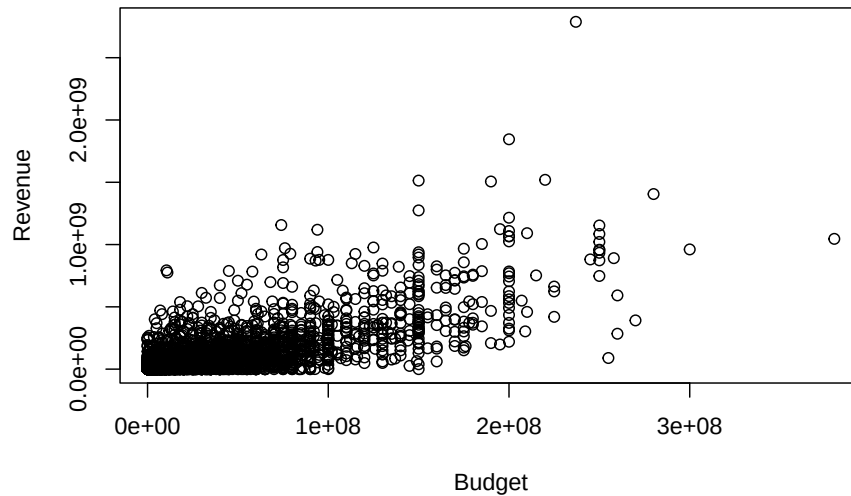
```
data_set = read.csv("tmdb_5000_movies.csv")
```

Suppose we are interested in studying how the budget of a movie is related to its revenue and we model the relationship as:

$$revenue_i = \beta_1 budget_i + \beta_2 + e_i$$

Before we run a regression, let's look at a scatter plot of the relevant variables:

```
plot(data_set$budget, data_set$revenue, xlab = "Budget", ylab = "Revenue")
```



Clearly, movies with higher budgets tend to have higher revenues. To estimate the regression coefficients, you can use the `lm()` command:

```
lm_result <- lm(formula = data_set$revenue~data_set$budget)
lm_summary <- summary(lm_result)
lm_summary
```

```
##
## Call:
## lm(formula = data_set$revenue ~ data_set$budget)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -653371282 -35365659  2250851   8486969 2097912654
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -2.630e+06  1.970e+06  -1.335    0.182
## data_set$budget  2.923e+00  3.940e-02  74.188 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 111200000 on 4801 degrees of freedom
```

```
## Multiple R-squared:  0.5341, Adjusted R-squared:  0.534
## F-statistic:  5504 on 1 and 4801 DF,  p-value: < 2.2e-16
```

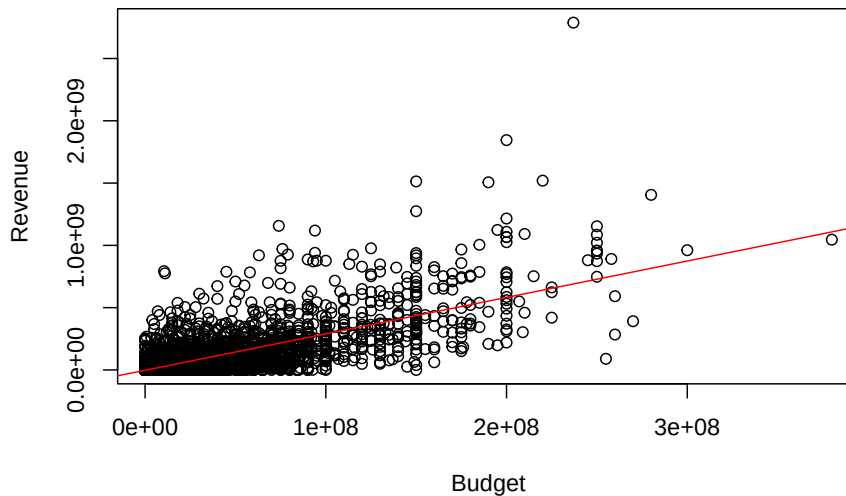
Think about how you can interpret the estimated slope coefficient.

We can also plot the regression line in the scatter plot:

```
b0 <- lm_summary$coefficients[1]
b1 <- lm_summary$coefficients[2]

## Plot all:
plot(y=data_set$revenue, x=data_set$budget,
     xlab="Budget",
     ylab="Revenue",
     main="Scatter plot and regression line")
abline(a=b0, b=b1, col="red")
```

Scatter plot and regression line



Let's now see if a higher budget also implies a higher rating:

```
lm_result_rating <- lm(data_set$vote_average~data_set$budget)
lm_summary_rating <- summary(lm_result_rating)
lm_summary_rating
```

```
##
## Call:
## lm(formula = data_set$vote_average ~ data_set$budget)
##
## Residuals:
```

```
##      Min      1Q  Median      3Q      Max
## -6.0224 -0.5131  0.1139  0.7462  3.9872
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    6.013e+00  2.108e-02 285.190 < 2e-16 ***
## data_set$budget 2.732e-09  4.215e-10   6.482 9.95e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.19 on 4801 degrees of freedom
## Multiple R-squared:  0.008676,    Adjusted R-squared:  0.00847
## F-statistic: 42.02 on 1 and 4801 DF,  p-value: 9.949e-11
```

Notice that the estimated slope coefficient is tiny. Does that mean that the rating is unrelated to the budget?

The budget is often huge. For a better interpretation of the coefficients, define the budget in millions as

```
data_set$budget_mil <- data_set$budget/1000000
```

look at summary statistics of the variables

```
summary(data_set$vote_average)
```

```
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.
##  0.000   5.600   6.200   6.092   6.800  10.000
```

```
summary(data_set$budget_mil )
```

```
##      Min. 1st Qu.  Median      Mean 3rd Qu.      Max.
##   0.00   0.79   15.00   29.05   40.00  380.00
```

and redo the analysis:

```
lm_result_rating <- lm(formula = data_set$vote_average~data_set$budget_mil)
lm_summary_rating <- summary(lm_result_rating)
lm_summary_rating
```

```
##
## Call:
## lm(formula = data_set$vote_average ~ data_set$budget_mil)
##
## Residuals:
##      Min      1Q  Median      3Q      Max
## -6.0224 -0.5131  0.1139  0.7462  3.9872
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept)          6.0128066  0.0210835 285.190 < 2e-16 ***
## data_set$budget_mil 0.0027325  0.0004215   6.482 9.95e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.19 on 4801 degrees of freedom
## Multiple R-squared:  0.008676,    Adjusted R-squared:  0.00847
## F-statistic: 42.02 on 1 and 4801 DF,  p-value: 9.949e-11
```

Do think that the budget has a large impact on the rating?

Of course, we are not limited to one regressors. To include three regressors, can use `lm(formula = Y~X1+X2+X3)`. We will return to regression analysis later in the semester.

7.3 Exercises VII

Exercise 1:

In the previous section, we looked at Monte Carlo simulations in a very simple setting. We can analyze analogous issues in other settings, such as the linear regression model. Also in this case, we have parameters (the regression coefficients) and estimators (the OLS estimators). The parameters are random and have certain known small and large sample properties that you discussed in the theoretical part of the class. Many other small sample properties, such as actual coverage rates of confidence intervals are unknown, but can be studied in Monte Carlo simulations.

Let

$$Y_i = \beta_1 X_{i1} + \beta_2 X_{i2} + \beta_3 + e_i, \quad i = 1, 2, \dots, n.$$

where

$$\begin{pmatrix} X_{i1} \\ X_{i2} \end{pmatrix} \sim N \left(\begin{pmatrix} 2 \\ -3 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix} \right)$$

and $(\beta_1, \beta_2, \beta_3) = (2, 1, -1)$. Let $e_i \sim U[-2, 2]$ and assume that e_i is independent of (X_{i1}, X_{i2}) . Assume that the data is a random sample. Let $(\hat{\beta}_1, \hat{\beta}_2, \hat{\beta}_3)$ be the OLS estimator of $(\beta_1, \beta_2, \beta_3)$.

For different values of n and ρ use simulations to calculate

- $E(\hat{\beta}_1)$,
- $Var(\hat{\beta}_1)$,
- the probability that β_1 is in a 95% confidence interval,
- the expected length of the confidence interval,
- the probability that we reject $H_0 : \beta_1 = 2$ using a two-sided t-test and a 1% significance level, and
- the probability that we reject $H_0 : \beta_1 = 1.8$ using a two-sided t-test and a 1% significance level.

Discuss the results.

How do you think the results will change if X_{i1} and e_i are correlated?

Exercise 2:

Start with the help file `?lm()` of the linear model - one of the functions most often used in R - and try to answer these questions:

- How do you estimate a model without a constant?
- In what setting could `na.action = na.exclude` be more useful to you than the function's default value?
- For a given `lm`-object, what does the function `summary` do?

Exercise 3:

Assume that $X \sim U[0, 1]$ and $Y \sim U[0, 1]$ are independent random variables. For different sample sizes ($n \in \{2, 5, 10, 100, 1000\}$) draw two i.i.d. samples of X and Y and regress Y on X for both samples. Using a Monte-Carlo simulation, approximate the probability that the two corresponding regression lines intersect on $[0, 1] \times [0, 1]$. Do the results depend on the sample size?

Chapter 8

Some notes on useful packages

8.1 The tidyverse family

"The tidyverse is an opinionated collection of R packages designed for data science. All packages share an underlying design philosophy, grammar, and data structures." The tidyverse family of R packages provides a comprehensive framework for many useful tools we need: from reading in data with readr [csv, tsv etc.] and haven [SPSS, Stata, SAS], wrangling it into the shape one desires with dplyr (mostly so called tidy-data) to visualization with ggplot2. These packages are not necessary to use R efficiently, but have a large following in the R community and often provide a more comfortable way to perform tedious tasks. A comprehensive guide to the tidyverse is provided in Hadley Wickham's book R for Data Science.

8.2 Tidymodels

A set of modeling packages built on the tidyverse architecture that can be used for a multitude of tasks. <https://www.tidymodels.org/>

8.3 lmtest

"A collection of tests, data sets, and examples for diagnostic checking in linear regression models. Furthermore, some generic tools for inference in parametric models are provided."

8.4 MASS

Collection of many useful functions that originated as a support package for a book on applied statistics with S (the predecessor language of R). One example is its function `mvrnorm` that enables its user to draw from a multivariate normal with a given mean vector and covariance matrix.

```
library(MASS)
my_sample <- mvrnorm(n = 100, mu = c(0,0), Sigma = matrix(data = c(1,0.5,0.5,1), nrow =
head(my_sample)
```

```
##           [,1]      [,2]
## [1,] -1.04001110 -1.1489202
## [2,] -1.16540797 -1.8931664
## [3,]  1.57256382  1.2056979
## [4,]  1.09718163 -0.8066595
## [5,] -0.02958764  0.5466597
## [6,]  0.96342122  0.2601426
```

8.5 AER - “Applied Econometrics with R”

Provides many useful function in the context of applied Econometrics. One prominent example is the function `ivreg` used for IV-estimation.

8.6 knitr

Formally an engine for dynamic report generation, knitr can be especially useful in your studies, if you want to use R in conjunction with markdown and output to different files. For example. this introduction is created from a bunch of Rmd files [R Markdown]. It can also be used to utilize R in a similar way to a Jupyter notebook, intermingling code and text freely.

8.7 Shiny

Used to build interactive web-apps with R. Somewhat more advanced, but very cool.

8.8 Faster Code

You might want to look at these packages:

- Rcpp
- parallel
- data.table
- future & future.apply

These can get complicated very quickly, so beware.